

Получение атрибутов в JavaScript

Метод `getAttribute`

Метод `getAttribute` считывает значение заданного атрибута у тега.

Синтаксис

```
1 элемент.getAttribute(имя атрибута);
```

Пример

Давайте выведем содержимое атрибута `value` элемента:

```
1 <input id="elem" value="abcde">

1 let elem = document.querySelector('#elem');
2 let value = elem.getAttribute('value');
3
4 console.log(value);
```

Результат выполнения кода:

```
1 'abcde'
```

№1

⊗jsPmAtGt

Дан элемент:

```
1 <input id="elem" value="text">
```

Получите значение его атрибута `value`.

№2

⊗jsPmAtGt

Дан элемент:

```
1 <input id="elem" class="www zzz">
```

Получите значение его атрибута `class`.

Установка атрибутов в JavaScript

Метод setAttribute

Метод `setAttribute` позволяет изменить значение заданного атрибута какого-либо тега.

Синтаксис

```
1 элемент.setAttribute(имя атрибута, новое значение);
```

Пример

Давайте элементу поменяем значение атрибута `value`:

```
1 <input id="elem" value="abcde">
2
1 let elem = document.querySelector('#elem');
2 elem.setAttribute('value', '!!!!');
```

HTML код станет выглядеть так:

```
1 <input id="elem" value="!!!!">
```

№1

⊗jsPmAtStM

Дан элемент:

```
1 <input id="elem">
```

Установите его атрибут `value` в значение `'text'`.

№2

⊗jsPmAtStM

Дан элемент:

```
1 <input id="elem">
```

Установите ему атрибут `class` в значение `'valid'`.

Удаление атрибутов в JavaScript

Метод removeAttribute

Метод `removeAttribute` удаляет заданный атрибут у какого-либо тега.

Синтаксис

```
1 элемент.removeAttribute(имя атрибута);
```

Пример

Давайте для элемента удалим атрибут `value`:

```
1 <input id="elem" value="abcde">

1 let elem = document.querySelector('#elem');
2 elem.removeAttribute('value');
```

HTML код станет выглядеть так (исчезнет атрибут `value`):

```
1 <input id="elem">
```

№ 1

⊗jsPmAtRm

Дан элемент:

```
1 <input id="elem" value="text">
```

Удалите у него атрибут `value`.

Проверка наличия атрибутов в JavaScript

Метод `hasAttribute`

Метод `hasAttribute` проверяет наличие заданного атрибута у элемента. Если атрибут есть - выведет `true`, если нет - выведет `false`.

Синтаксис

```
1 элемент.hasAttribute(имя атрибута);
```

Пример

Давайте проверим наличие атрибута `value` у элемента:

```
1 <input id="elem" value="abcde">

1 let elem = document.querySelector('#elem');
2 console.log(elem.hasAttribute('value'));
```

Результат выполнения кода:

```
1 true
```

Пример

А теперь атрибута value нету:

```
1 <input id="elem">

1 let elem = document.querySelector('#elem');
2 console.log(elem.hasAttribute('value'));
```

Результат выполнения кода:

```
1 false
```

№ 1

⊗jsPmAtCh

Дан элемент:

```
1 <input id="elem" value="text">
```

Проверьте наличие у него атрибута value.

Пользовательские атрибуты в JavaScript

В HTML разрешено добавлять свои, пользовательские атрибуты тегам. Такие атрибуты должны начинаться с data-, а затем должно идти любое название атрибута, которое вам удобно.

Пользовательские атрибуты могут быть использованы огромным количеством различных способов. Многие из этих способов мы изучим далее в учебнике, а еще больше вы сможете в дальнейшем изобрести самостоятельно.

Обращение к таким атрибутам устроено не стандартным образом. Нельзя просто обратиться к одноименному свойству элемента, как мы делали это раньше, а следует использовать специальное свойство dataset, после которого через точку пишется имя атрибута без data-. Например, если наш атрибут называется data-test, то для обращения к нему мы будем писать elem.dataset.test, где elem - переменная с нашим элементом.

Давайте посмотрим на примере. Пусть у нас дан вот такой элемент:

```
1 <div id="elem" data-num="1000"></div>
```

Выведем на экран значение его атрибута data-num:

```
1 let elem = document.querySelector('#elem');
2 console.log(elem.dataset.num); // выведет 1000
```

А теперь присвоим этому атрибуту другое значение:

```
1 let elem = document.querySelector('#elem');
2 elem.dataset.num = 123;
```

№1

⊗jsPmAtDt

Дан следующий код:

```
1 <div id="elem" data-text="str">text</div>
```

Сделайте так, чтобы по клику на див в конец его текста добавилось содержимое его атрибута data-text.

№2

⊗jsPmAtDt

Даны дивы, содержащие в атрибуте data-num свой порядковый номер:

```
1 <div data-num="1">text</div>
2 <div data-num="2">text</div>
3 <div data-num="3">text</div>
4 <div data-num="4">text</div>
5 <div data-num="5">text</div>
```

Сделайте так, чтобы по клику на любой из дивов ему в конец записывался его порядковый номер.

№3

⊗jsPmAtDt

Дана кнопка. Сделайте так, чтобы эта кнопка считала количество кликов по ней, записывая их в какой-нибудь пользовательский атрибут. Пусть по клику на другую кнопку на экран выводится, сколько кликов было сделано по первой кнопке.

№4

⊗jsPmAtDt

Дан инпут:

```
1 <input id="elem" data-length="5">
```

В этом инпуте в атрибуте data-length содержится количество символов, которое нужно ввести в инпут. Сделайте так, чтобы по потере фокуса, если количество введенных символов не совпадает с заданным, выводилось сообщение об этом.

№5

⊗jsPmAtDt

Дан инпут:

```
1 <input id="elem" data-min="5" data-max="10">
```

В этом инпуте атрибуты data-min и data-max содержат диапазон. Сделайте так, чтобы по потере фокуса, если количество введенных символов не попадает в этот диапазон, выводилось сообщение об этом.

Имена атрибутов с дефисами в JavaScript

Пользовательские атрибуты могут содержать дефисы в своем названии, например, вот так:

```
1 <div id="elem" data-my-test="1000"></div>
```

Для обращения к таким атрибутам следует использовать camelCase:

```
1 let elem = document.querySelector('#elem');
2 console.log(elem.dataset.myTest);
```

№ 1

⊗ jsPmAtDHN

Дан следующий код:

```
1 <div id="elem" data-product-price="1000" data-product-amount="5">
2   товар яблоки
3 </div>
```

Сделайте так, чтобы по клику на див в конец его текста добавлялась стоимость покупки, равная цене, умноженной на количество.

Обращение к атрибутам через методы в JavaScript

К пользовательским атрибутам можно также обращаться с помощью методов типа `getAttribute`, в этом случае следует писать полное название атрибута:

```
1 <div id="elem" data-num="1000" data-my-num="2000"></div>

1 let elem = document.querySelector('#elem');
2
3 console.log(elem.getAttribute('data-num')); // выведет 1000
4 console.log(elem.getAttribute('data-my-num')); // выведет 2000
```

№ 1

⊗ jsPmAtDM

Даны абзацы. Переберите их циклом и каждому абзацу в атрибут `data-num` запишите порядковый номер этого абзаца. Используйте метод `setAttribute`.

Манипулирование массивом CSS классов в JavaScript

Свойство `classList`

Свойство `classList` содержит псевдомассив CSS классов элемента, а также позволяет добавлять и удалять классы элемента, проверять наличие определенного класса среди классов элемента.

Речь идет об атрибуте `class`, внутри которого можно писать несколько классов через пробел, например `www ggg zzz`. С помощью `classList` можно удалить, к примеру, класс `ggg`, не затронув остальные классы.

Синтаксис

```
1 элемент.classList;
```

Пример . Количество классов

Узнаем количество классов элемента:

```
1 <p id="elem" class="www ggg zzz"></p>

1 let elem = document.querySelector('#elem');
2
3 let length = elem.classList.length;
4 console.log(length);
```

Результат выполнения кода:

```
1 3
```

Пример . Перебираем классы

Выведем по очереди классы элемента:

```
1 <p id="elem" class="www ggg zzz"></p>

1 let elem = document.querySelector('#elem');
2 let classNames = elem.classList;
3
4 for (let className of classNames) {
5   console.log(className);
6 }
```

Результат выполнения кода:

```
1 'www'
2 'ggg'
3 'zzz'
```

№1

⊗jsPmAtCAr

Дан элемент:

```
1 <p id="elem" class="www ggg zzz"></p>
```

Узнайте количество его классов.

№2

⊗jsPmAtCAr

Дан элемент:

```
1 <p id="elem" class="www ggg zzz"></p>
```

Переберите в цикле его классы.

Добавление CSS классов в JavaScript

Метод add объекта classList

Метод add объекта `classList` позволяет добавлять CSS классы элементу.

Синтаксис

```
1 element.classList.add(class);
```

Пример

Добавим элементу класс kkk:

```
1 <p id="elem" class="www ggg zzz"></p>

1 let elem = document.querySelector('#elem');
2 elem.classList.add('kkk');
```

Результат выполнения кода:

```
1 <p id="elem" class="www ggg zzz kkk"></p>
```

Пример

Добавим элементу класс zzz, который уже есть в элементе - ничего не произойдет, так как дубли классов не добавляются:

```
1 <p id="elem" class="www ggg zzz"></p>

1 let elem = document.querySelector('#elem');
2 elem.classList.add('zzz');
```

Результат выполнения кода:

```
1 <p id="elem" class="www ggg zzz"></p>
```

№ 1

©jsPmAtCAAd

Дан элемент:

```
1 <p id="elem" class="www ggg zzz"></p>
```

Добавьте ему класс xxx.

Удаление CSS классов в JavaScript

Метод remove объекта classList

Метод remove объекта `classList` удаляет заданный CSS класс элемента.

Синтаксис

```
1 element.classList.contains(class);
```


Пример

Удалим класс ggg:

```
1 <p id="elem" class="www ggg zzz"></p>

1 let elem = document.querySelector('#elem');
2 elem.classList.remove('ggg');
```

Результат выполнения кода:

```
1 <p id="elem" class="www zzz"></p>
```

№ 1

⊗jsPmAtCR

Дан элемент:

```
1 <p id="elem" class="www ggg zzz"></p>
```

Удалите у него класс www и класс zzz.

Проверка CSS классов в JavaScript

Метод contains объекта classList

Метод contains объекта `classList` проверяет наличие CSS класса элемента.

Синтаксис

```
1 element.classList.contains(class);
```

Пример

Проверим, есть ли у элемента класс ggg:

```
1 <p id="elem" class="www ggg zzz"></p>

1 let elem = document.querySelector('#elem');
2
3 let contains = elem.classList.contains('ggg');
4 console.log(contains);
```

Результат выполнения кода:

```
1 true
```

№ 1

⊗jsPmAtCCh

Дан элемент:

```
1 <p id="elem" class="www ggg zzz"></p>
```

Проверьте наличие у него класса ggg.

Чередование CSS классов в JavaScript

Метод toggle объекта classList

Метод toggle объекта classList чередует заданный CSS класс элемента: добавляет класс, если его нет и удаляет, если есть.

Синтаксис

```
1 element.classList.contains(class);
```

Пример

В данном примере при использовании метода toggle класс zzz удалится, так как он уже есть в элементе:

```
1 <p id="elem" class="www ggg zzz"></p>

1 let elem = document.querySelector('#elem');
2 elem.classList.toggle('zzz');
```

Результат выполнения кода:

```
1 <p id="elem" class="www ggg"></p>
```

Пример

В данном примере при использовании метода toggle класс zzz добавится, так как его нету в элементе:

```
1 <p id="elem" class="www ggg"></p>

1 let elem = document.querySelector('#elem');
2 elem.classList.toggle('zzz');
```

Результат выполнения кода:

```
1 <p id="elem" class="www ggg zzz"></p>
```

№ 1

⊗jsPmAtCT

Дан элемент. Добавьте ему класс www, если его нет и удалите - если есть.

